实验2 软中断通信

- 实验目的
  - 本实验要求学生了解什么是信号,掌握软中断的基本原理;掌握中断信号的使用、进程的创建以及系统计时器的使用
  - 通过对本实验的学习，学生能够学会进程的创建方法，更能加深对Linux中的信号机制的认识，并会使用软中断信号来实现进程间的通信。

实验内容(1)

- 观察并理解样例程序test2.c的行为
  - 父进程接受到软中断信号（SIGQUIT）后，向其子进程分别发送整数值为16的软中断信号，子进程获得对应软中断信号后，终止运行
  - 父进程调用wait()函数等待子进程终止，然后自我终止
  - 由父进程创建一个子进程，通过终端输入Crtl+\组合键向父进程发送SIGQUIT软中断信号

实验内容(2)

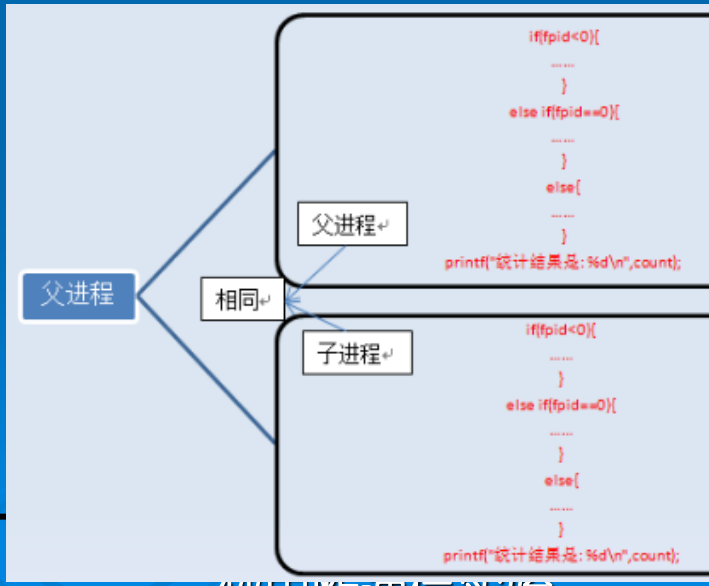
- 编程实现以下功能
  - 由一个父进程创建两个子进程，之后通过终端输入Crtl+\组合键向父进程发送软中断信号，终止两个子进程以及父进程
  - 由一个父进程创建一个子进程，之后该子进程再创建一个孙进程，通过终端输入Crtl+\组合键向父进程发送软中断信号，依次终止孙进程、子进程、父进程

Linux进程控制-进程创建

1.进程创建：fork()函数

- 一个进程，包括代码、数据和分配给进程的资源。
- fork()函数通过系统调用创建一个与原来进程几乎完全相同的进程。fork创建的新进程被称为子进程（child process）。该函数被调用一次，但返回两次。两次返回的区别是子进程的返回值是0，而父进程的返回值则是新进程（子进程）的进程id。

- int getpid() 返回当前进程标识
- int getppid() 返回父进程标识



Linux进程控制-进程创建

- fork之后，操作系统会复制一个与父进程完全相同的子进程，虽说是父子关系，但是在操作系统看来，他们更像兄弟关系，这2个进程共享代码空间，但是数据空间是互相独立的，子进程数据空间中的内容是父进程的完整拷贝，指令指针也完全相同，子进程拥有父进程当前运行到的位置
- 子进程虽然继承了父进程的一切数据，但它一开始运行，就和父进程分开，子进程拥有自己的进程号、资源使用和计时器等，与父进程不再共享任何数据
- 如果子进程和父进程需要通信，则只能通过进程间(IPC)的通信方式来实现

Linux进程控制-进程终止和等待

2. 进程终止：exit()

exit()函数在调用exit系统之前要检查文件的打开情况，把文件缓冲区的内容写回文件,然后再终止进程。

3. 进程等待：wait()

进程一旦调用了wait就立即阻塞自己，由wait自动分析是否当前进程的某个子进程已经退出，如果让它找到了这样一个已经变成僵尸的子进程，wait就会收集这个子进程的信息，并把它彻底销毁后返回；如果没有找到这样一个子进程，wait就会一直阻塞在这里，直到有一个出现为止。

Linux进程控制-锁定文件

4. 锁定文件：lockf (files, function, size)

files是文件描述符；function是锁定和解锁：1表示锁定，0表示解锁。size是锁定或解锁的字节数，为0表示从文件的当前位置到文件尾。

eg. lockf(1,1,0) 加锁，第一个参数为stdout（标准输出设备的描述符）

lockf(1,0,0)解锁

```c
int main()
{
    int p1;
    while ((p1 = fork()) == -1);
    if (p1>0) //if p1 is parent process
    {
        printf("parent process is %d \n", getpid());
        wait_mark=1;
        signal(SIGQUIT, stop);
        waiting();
        kill(p1,16); //send signal 16 to end the process p1
        wait(0); //waiting for the ending of p1
        printf("parent process is killed!\n");
        exit(0); //quit from the parent process

    }
    else //if p1 is child process
    {

        printf("child process %d is created by the parent %d \n", getpid(), getppid());
        signal(SIGQUIT,SIG_IGN);
        wait_mark=1;
        signal(16, stop);
        waiting();
        lockf(1,1,0);
        printf("child process %d is killed by parent %d \n", getpid(), getppid());
        lockf(1,0,0);
        exit(0); // p1 quit
    }
    return 0;
}
void waiting()
```

进程间通信方式

1. 匿名管道(Anonymous Pipe)及有名管道(named pipe)
   匿名管道可用于具有亲缘关系进程间的通信，有名管道还允许无亲缘关系进程间的通信
2. 信号(Signal)
   信号用于通知接受进程有某种事件发生，除用于进程间通信外，进程还可以发送信号给进程本身。
3. 报文(Message)队列
   消息的链接表，包括Posix消息队列systemV消息队列。
4. 共享内存(Shared Memory)
   使得多个进程可以访问同一块内存空间
5. 信号量(semaphore)
   主要作为进程间以及同一进程不同线程之间的同步手段。
6. 套接口(Socket)
   可用于不同机器之间的进程间通信。

信号

信号本质：

1. 软中断信号(signal，又简称为信号)用来通知进程发生了异步事件。在软件层次上是对中断机制的一种模拟。在原理上，一个进程收到一个信号与处理器收到一个中断请求的处理类似
2. 进程之间可以互相通过系统调用kill发送软中断信号。

信号事件的发生有两个来源：

1. 硬件来源(例如用户按下了键盘或者其它硬件故障)
2. 软件来源，最常用发送信号的系统调用函数有
   - 发生信号函数 kill, raise
   - 捕捉信号函数 alarm, pause
   - 处理信号函数 signal, sigaction

Linux信号表

Linux支持POSIX标准信号和实时信号。下面给出Linux Signal的简表，详细细节可以查看man 7 signal。

信号	取值	默认动作	含义（发出信号的原因）
SIGHUP	1	Term	终端的挂断或进程死亡
SIGINT	2	Term	来自键盘的中断信号
SIGQUIT	3	Core	来自键盘的离开信号
SIGILL	4	Core	非法指令
SIGABRT	6	Core	来自abort的异常信号
SIGFPE	8	Core	浮点例外
SIGKILL	9	Term	杀死
SIGSEGV	11	Core	段非法错误(内存引用无效)
SIGPIPE	13	Term	管道损坏：向一个没有读进程的管道写数据
SIGALRM	14	Term	来自alarm的计时器到时信号
SIGTERM	15	Term	终止
SIGUSR1	30,10,16	Term	用户自定义信号1
SIGUSR2	31,12,17	Term	用户自定义信号2
SIGCHLD	20,17,18	Ign	子进程停止或终止
SIGCONT	19,18,25	Cont	如果停止，继续执行
SIGSTOP	17,19,23	Stop	非来自终端的停止信号
SIGTSTP	18,20,24	Stop	来自终端的停止信号
SIGTTIN	21,21,26	Stop	后台进程读终端
SIGTTOU	22,22,27	Stop	后台进程写终端

信号处理方法

三类处理方法：

1. 类似中断的处理程序，对于需要处理的信号，进程可以指定处理函数，由该函数来处理。
2. 忽略某个信号，对该信号不做任何处理，就象未发生过一样。
3. 对该信号的处理保留系统的默认值，这种缺省操作，对大部分的信号的缺省操作是使得进程终止。进程通过系统调用signal来指定进程对某个信号的处理行为

软中断信号相关的系统调用

发送信号函数： int kill(pid, sig)

pid是一个或一组进程的标识符，参数sig是要发送的软中断信号。

（1）pid>0时，核心将信号发送给进程pid。
（2）pid=0时，核心将信号发送给与发送进程同组的所有进程。
（3）pid=-1时，核心将信号发送给所有用户标识符真正等于发送进程的有效用户标识号的进程。

接收信号函数：signal(sig，function)

参数sig是接收到的的软中断信号，funcntion预置对信号的处理方式。

（1）function=1时，进程对sig类信号不予理睬，SIG_IGN
（2）function=0时，缺省值，进程在收到sig信号后应终止自己，SIG_DEL；
（3）function为非0，非1类整数时，function的值作为信号处理程序的指针。

```c
int main()
{
    int p1;
    while ((p1 = fork()) == -1);
    if (p1>0) //if p1 is parent process
    {
        printf("parent process is %d \n", getpid());
        wait_mark=1;
        signal(SIGQUIT, stop);
        waiting();
        kill(p1,16); //send signal 16 to end the process p1
        wait(0); //waiting for the ending of p1
        printf("parent process is killed!\n");
        exit(0); //quit from the parent process

    }
    else //if p1 is child process
    {

        printf("child process %d is created by the parent %d \n", getpid(), getppid());
        signal(SIGQUIT,SIG_IGN);
        wait_mark=1;
        signal(16, stop);
        waiting();
        lockf(1,1,0);
        printf("child process %d is killed by parent %d \n", getpid(), getppid());
        lockf(1,0,0);
        exit(0); // p1 quit
    }
    return 0;
}
void waiting()
```

操作方法和实验步骤(1)

实验 2-1：编译并运行程序test2.c，当按下Crtl+\组合键时，打印出子进程结束的信息，最后打印出父进程结束的信息

- 创建一个子进程
- 子进程分别等待信号16，如果收到信号则显示结束信息，并发出结束信号
- 父进程等待SIGQUIT信号，如果收到信号则向子进程发送信号16，接着等子进程结束，如果都结束了则显示结束信息，并退出进程

操作方法和实验步骤(2)

实验2-2：编写程序，实现两种情况的三个进程通信

- 由一个父进程创建两个子进程，之后通过终端输入Crtl+\组合键向父进程发送软中断信号，终止两个子进程以及父进程。
- 由一个父进程创建一个子进程，之后该子进程再创建一个孙进程，通过终端输入Crtl+\组合键向父进程发送软中断信号，依次终止孙进程、子进程、父进程。